

VOLUME 2

Déploiement · Base de Données · Secrets

Déploiement complet d'une application, accès BDD depuis le cluster et en local, gestion des secrets Azure Key Vault



Déploiement
Application



Base de Données
Azure



Accès Local
(Port-Forward)



Secrets Key Vault



External Secrets
Operator

① Déploiement Complet d'une Application sur AKS/OpenShift

Walkthrough pas-à-pas d'un déploiement production-ready : création du Namespace avec ResourceQuota, ConfigMap pour la configuration non-sensible, référence aux secrets Azure Key Vault, Deployment avec rolling update / probes de santé / securityContext durci, Service ClusterIP, et HPA pour l'autoscaling automatique. Chaque manifest YAML est expliqué ligne à ligne.

Exemple complet : application Node.js/Python avec base de données PostgreSQL, secrets Azure Key Vault, Ingress HTTPS.

Fichier / Étape	YAML / Commande	Explication
1. Namespace & Quotas	apiVersion: v1 kind: Namespace metadata: name: mon-app labels: app.kubernetes.io/name: mon-app environment: production --- apiVersion: v1 kind: ResourceQuota metadata: name: mon-app-quota namespace: mon-app spec: hard: requests.cpu: '4' requests.memory: 8Gi limits.cpu: '8' limits.memory: 16Gi pods: '20'	Isoler l'application dans son namespace Quotas : éviter qu'une app monopolise le cluster Labels : utilisés par les sélecteurs et Network Policies
2. ConfigMap	apiVersion: v1 kind: ConfigMap metadata: name: app-config namespace: mon-app data: APP_ENV: 'production' DB_HOST: 'postgres-svc.mon-app.svc.cluster.local' DB_PORT: '5432' DB_NAME: 'appdb' LOG_LEVEL: 'info' REDIS_HOST: 'redis-svc.mon-app.svc.cluster.local'	Configuration non-sensible uniquement Ne jamais mettre de mots de passe ici DB_HOST = DNS interne Kubernetes (résolu par CoreDNS)
3. Secret (référence AKV)	# Option A : Secret K8s classique (non recommandé prod) apiVersion: v1 kind: Secret metadata: name: app-secrets namespace: mon-app type: Opaque data: DB_PASSWORD: <base64> # éphémère API_KEY: <base64> # Option B : External Secret (recommandé - voir section Secrets) apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: app-secrets namespace: mon-app spec: refreshInterval: 1h secretStoreRef: name: azure-keyvault-store kind: ClusterSecretStore target: name: app-secrets data: - secretKey: DB_PASSWORD remoteRef: key: app-db-password - secretKey: API_KEY remoteRef: key: app-api-key	Option B = synchronisation depuis Azure Key Vault Rotation automatique toutes les heures Le pod voit toujours des secrets Kubernetes normaux → Détails complets en Section ④

Fichier / Étape	YAML / Commande	Explication
4. Deployment	<pre>apiVersion: apps/v1 kind: Deployment metadata: name: mon-app namespace: mon-app labels: app: mon-app spec: replicas: 3 selector: matchLabels: app: mon-app strategy: type: RollingUpdate rollingUpdate: maxUnavailable: 1 maxSurge: 1 template: metadata: labels: app: mon-app spec: serviceAccountName: mon-app-sa containers: - name: app image: acrprod.azurecr.io/mon-app:v1.2.0 ports: - containerPort: 3000 envFrom: - configMapRef: name: app-config - secretRef: name: app-secrets resources: requests: cpu: 250m memory: 256Mi limits: cpu: 500m memory: 512Mi livenessProbe: httpGet: path: /health port: 3000 initialDelaySeconds: 30 periodSeconds: 10 readinessProbe: httpGet: path: /ready port: 3000 initialDelaySeconds: 10 periodSeconds: 5 securityContext: runAsNonRoot: true runAsUser: 1000 readOnlyRootFilesystem: true allowPrivilegeEscalation: false</pre>	replicas: 3 → haute disponibilité RollingUpdate → zéro downtime envFrom → inject toute la ConfigMap/Secret resources requests/limits → obligatoire pour HPA livenessProbe → redémarre si mort readinessProbe → retire du LB si non prêt securityContext → hardening sécurité
5. Service	<pre>apiVersion: v1 kind: Service metadata: name: mon-app-svc namespace: mon-app spec: selector: app: mon-app ports: - name: http protocol: TCP port: 80 targetPort: 3000 type: ClusterIP --- # Service interne pour la BDD apiVersion: v1 kind: Service metadata: name: postgres-svc namespace: mon-app spec: type: ExternalName externalName: mon-postgres.postgres.database.azure.com</pre>	ClusterIP : accessible seulement dans le cluster ExternalName : alias DNS vers Azure Database Pas de ClusterIP → trafic direct vers Azure DB via DNS
6. HPA	<pre>apiVersion: autoscaling/v2 kind: HorizontalPodAutoscaler metadata: name: mon-app-hpa namespace: mon-app spec: scaleTargetRef: apiVersion: apps/v1 kind: Deployment name: mon-app minReplicas: 3 maxReplicas: 20 metrics: - type: Resource resource: name: cpu target: type: Utilization averageUtilization: 70 - type: Resource resource: name: memory target: type: Utilization averageUtilization: 80 behavior: scaleDown: stabilizationWindowSeconds: 300</pre>	Scale automatique entre 3 et 20 replicas StabilizationWindow : évite le flapping (yo-yo) Scale down lent (5 min) pour éviter les interruptions Nécessite metrics-server installé
7. Appliquer & Vérifier	<pre># Appliquer tous les manifests kubectl apply -f ./k8s/ -n mon-app # ou kubectl apply -k ./kustomize/overlays/prod # Suivre le déploiement kubectl rollout status deploy/mon-app -n mon-app # Vérifier les pods kubectl get pods -n mon-app -w # Vérifier les events kubectl get events -n mon-app --sort-by='.lastTimestamp' # Logs en temps réel kubectl logs -f deploy/mon-app -n mon-app # Rollback si problème kubectl rollout undo deploy/mon-app -n mon-app</pre>	apply -f dossier : applique tous les YAML kustomize : gestion des environnements (dev/staging/prod) -w : watch (mise à jour en temps réel) rollout undo : revient à la version précédente

② Base de Données Azure — Connexion depuis le Cluster

Comment connecter des pods Kubernetes aux bases de données managées Azure (PostgreSQL Flexible Server, Azure SQL, Cosmos DB) en toute sécurité : création via Azure CLI, mise en place d'un Private Endpoint pour isoler la BDD dans le VNet, utilisation d'un Service ExternalName comme alias DNS interne, et exécution des migrations via un Job Kubernetes avant le déploiement applicatif.

Service Azure	Configuration Kubernetes	Commandes & Notes
Azure Database for PostgreSQL (Flexible Server)	<pre># 1. Créer la BDD Azure az postgres flexible-server create \ --resource-group rg-prod \ --name mon-postgres \ --location eastus \ --admin-user pgadmin \ --admin-password \$env:PG_PASSWORD \ --sku-name Standard_D2s_v3 \ --version 15 \ --public-access None # 2. Configurer Private Endpoint (AKS vers BDD) az network private-endpoint create \ --resource-group rg-prod \ --name pe-postgres \ --vnet-name aks-vnet \ --subnet aks-subnet \ --private-connection-resource-id \ \$(az postgres flexible-server show -g rg-prod -n mon-postgres --query id -o tsv) \ --group-id postgresqlServer \ --connection-name postgres-connection # 3. Service ExternalName dans K8s apiVersion: v1 kind: Service metadata: name: postgres-svc namespace: mon-app spec: type: ExternalName externalName: mon-postgres.postgres.database.azure.com</pre>	Private Endpoint = connexion privée via VNet Pas d'exposition internet de la BDD Service ExternalName = alias DNS K8s Les pods se connectent via postgres-svc:5432
Azure SQL Database	<pre># Créer Azure SQL az sql server create \ --resource-group rg-prod \ --name sql-mon-serveur \ --admin-user sqladmin \ --admin-password \$env:SQL_PASSWORD az sql db create \ --resource-group rg-prod \ --server sql-mon-serveur \ --name appdb \ --service-objective S3 # Private Endpoint az network private-endpoint create \ --resource-group rg-prod \ --name pe-sql \ --vnet-name aks-vnet \ --subnet aks-subnet \ --private-connection-resource-id \ \$(az sql server show -g rg-prod -n sql-mon-serveur --query id -o tsv) \ --group-id sqlServer \ --connection-name sql-connection # Connection string dans Secret K8s kubectl create secret generic sql-conn \ --from-l iteral=connectionString='Server=sql-mon-serveur.database.windows. net;Database=appdb;User Id=sqladmin;Password=...' \ -n mon-app</pre>	Utiliser Managed Identity pour éviter mots de passe az sql server ad-admin set \ --resource-group rg-prod \ --server sql-mon-serveur \ --object-id Connection string dans Secret ou AKV
Azure Cosmos DB	<pre># Créer Cosmos DB az cosmosdb create \ --resource-group rg-prod \ --name cosmos-mon-app \ --kind MongoDB \ --locations regionName=eastus # Récupérer la connection string \$connStr = az cosmosdb keys list \ --resource-group rg-prod \ --name cosmos-mon-app \ --type connection-strings \ --query connectionStrings[0].connectionString -o tsv # Stocker dans Key Vault az keyvault secret set \ --vault-name kv-mon-app \ --name cosmos-connection-string \ --value \$connStr</pre>	Cosmos DB = NoSQL multi-modèle (MongoDB API, SQL API...) Connection string → directement dans Key Vault Récupérée via External Secrets Operator Pas de Private Endpoint requis si réseau de service)

Service Azure	Configuration Kubernetes	Commandes & Notes
Migration DB depuis le cluster (Job Kubernetes)	<pre>apiVersion: batch/v1 kind: Job metadata: name: db-migrate namespace: mon-app spec: template: spec: restartPolicy: OnFailure containers: - name: migrate image: acrprod.azurecr.io/mon-app:v1.2.0 command: ['npm', 'run', 'migrate'] envFrom: - configMapRef: name: app-config - secretRef: name: app-secrets backoffLimit: 3 --- # Lancer et surveiller kubectl apply -f migrate-job.yaml kubectl wait --for=condition=complete job/db-migrate -n mon-app --timeout=120s kubectl logs job/db-migrate -n mon-app</pre>	Job = tâche à exécution unique (pas un Service) backoffLimit : réessaie 3 fois en cas d'échec Attendre la complétion avant le Deployment Dans CI/CD : step séparé avant le déploiement de l'app

③ Accès à la Base de Données depuis le Poste Local

Six méthodes pour se connecter à une base de données Azure depuis un poste développeur ou DBA, classées par complexité : port-forward kubectl (debug rapide), pod proxy socat (BDD accessible uniquement via VNet), Azure VPN Point-to-Site (accès permanent pour toute l'équipe), Azure Bastion + SSH tunnel (sécurisé sans VPN), kubectl exec (accès in-cluster), et script PowerShell réutilisable pour DBeaver/DataGrip.

Méthode	Commandes PowerShell / kubectl	Cas d'usage & Limitations
Port-Forward kubectl (méthode simple)	# Forwarder le service BDD kubectl port-forward svc/postgres-svc 5432:5432 -n mon-app # En arrière-plan (PowerShell) \$job = Start-Job -ScriptBlock { kubectl port-forward svc/postgres-svc 5432:5432 -n mon-app } # Se connecter avec psql ou DBeaver psql -h localhost -p 5432 -U pgadmin -d appdb # Stopper Stop-Job \$job; Remove-Job \$job	■ Simple, pas de config réseau ■ Idéal pour debug rapide ■ Connexion tunnel (non persistante) ■ Pas d'accès si pas de kubeconfig local
Port-Forward via pod proxy (quand ExternalName)	# Si postgres-svc est ExternalName (Azure DB) # Créer un pod netshoot temporaire kubectl run netshoot --rm -it \ --image=nicolaka/netshoot \ --restart=Never \ -n mon-app \ -- bash # Dans le pod : tester la connectivité psql -h postgres-svc -U pgadmin -d appdb # Depuis local : forwarder via pod intermédiaire kubectl run pg-proxy \ --image=alpine/socat \ --restart=Never \ -n mon-app \ -- socat TCP-LISTEN:5432,fork TCP:mon-postgres.postgres.database.azure.com:5432 kubectl port-forward pod/pg-proxy 5432:5432 -n mon-app	Utile quand la BDD Azure n'est accessible que via VNet le pod socat agit comme tunnel Supprimer le pod après utilisation ! Alternative : utiliser Azure Bastion + SSH tunnel
Azure VPN / Private Link (accès permanent)	# Option 1 : Azure VPN Gateway az network vnet-gateway create \ --resource-group rg-prod \ --name vpn-gw \ --vnet aks-vnet \ --gateway-type Vpn \ --sku VpnGw1 # Option 2 : Point-to-Site VPN (poste local → Azure VNet) # → Configurer dans Azure Portal : # VPN Gateway → Point-to-site configuration # Télécharger le client VPN # Se connecter → accès direct à 10.0.0.x # Une fois connecté au VPN : psql -h mon-postgres.postgres.database.azure.com \ -U pgadmin -d appdb -p 5432	■ Accès direct et permanent ■ Toutes les équipes peuvent se connecter ■ Coût de la VPN Gateway (~150€/mois) ■ Config initiale plus complexe Idéal pour équipes dev/DBA

Méthode	Commandes PowerShell / kubectl	Cas d'usage & Limitations
Azure Bastion + SSH Tunnel (sécurisé sans VPN)	<pre># 1. Créer Azure Bastion az network bastion create \ --resource-group rg-prod \ --name bastion-prod \ --vnet-name aks-vnet \ --public-ip-address bastion-pip # 2. Tunnel SSH via Bastion vers une VM jumpbox dans le VNet az network bastion tunnel \ --name bastion-prod \ --resource-group rg-prod \ --target-resource-id <vm-jumpbox-id> \ --resource-port 22 \ --port 2222 # 3. SSH tunnel depuis local (dans un autre terminal) ssh -L 5432:mon-postgres.postgres.database.azure.com:5432 \ -p 2222 azureuser@localhost # 4. Connexion BDD via le tunnel psql -h localhost -p 5432 -U pgadmin -d appdb</pre>	■ Sécurisé (pas de port public) ■ Pas de VPN Gateway ■ Nécessite une VM jumpbox dans le VNet Azure Bastion = proxy SSH managé par Azure Idéal pour accès ponctuel admin/DBA
kubectl exec direct (accès in-cluster)	<pre># Exec dans le pod de l'application kubectl exec -it \ \$(kubectl get pod -n mon-app -l app=mon-app -o jsonpath='{.items[0].metadata.name}') \ -n mon-app -- /bin/sh # Depuis le pod, se connecter à la BDD psql -h postgres-svc -U pgadmin -d appdb # Exécuter une requête directement kubectl exec -n mon-app deploy/mon-app -- \ psql -h postgres-svc -U pgadmin -d appdb \ -c 'SELECT version();' # Dump de la BDD depuis un pod kubectl exec -n mon-app deploy/mon-app -- \ pg_dump -h postgres-svc -U pgadmin appdb > dump.sql</pre>	■ Utilise les credentials déjà dans le pod ■ Pas de config réseau local ■ Accès dépend des permissions RBAC Ne pas utiliser en production pour des modifications Bon pour déboguer le problème réseau/credentials
DBeaver / DataGrip via port-forward automatisé	<pre># Script PowerShell pour port-forward facile function Connect-KubeDB { param([string]\$Namespace = 'mon-app', [string]\$Service = 'postgres-svc', [int]\$LocalPort = 5432, [int]\$RemotePort = 5432) Write-Host "Starting port-forward: localhost:\$LocalPort -> \$Service:\$RemotePort" \$pf = Start-Process kubectl \ -ArgumentList "port-forward svc/\$Service \${LocalPort}:\${RemotePort} -n \$Namespace" \ -PassThru -WindowStyle Hidden Write-Host "Port-forward PID: \$(\$pf.Id)" Write-Host "Connect to: localhost:\$LocalPort" return \$pf } # Usage \$pf = Connect-KubeDB -Namespace mon-app -Service postgres-svc # ... utiliser DBeaver/DataGrip sur localhost:5432 ... \$pf.Kill() # Stopper</pre>	Script réutilisable pour l'équipe DBeaver : New Connection → PostgreSQL → localhost:5432 DataGrip : identique Ajouter à votre profil PowerShell \$PROFILE pour usage fréquent

④ Gestion des Secrets — Azure Key Vault & External Secrets Operator

Stratégie complète pour ne jamais stocker de secrets en clair dans Kubernetes ou Git : création d'un Azure Key Vault avec RBAC, activation de Workload Identity sur AKS (pods s'authentifient via identité managée sans mot de passe statique), deux méthodes d'injection (CSI Secret Store Driver pour montage en volume, External Secrets Operator pour synchronisation en variables d'environnement), politiques de rotation et audit, et Sealed Secrets pour les workflows GitOps.

Étape / Composant	Commande / YAML	Description
Créer Azure Key Vault	<pre># Créer le Key Vault az keyvault create \ --resource-group rg-prod \ --name kv-mon-app \ --location eastus \ --enable-rbac-authorization true \ --enable-soft-delete true \ --retention-days 90 # Ajouter des secrets az keyvault secret set \ --vault-name kv-mon-app \ --name db-password \ --value 'MonSuperMotDePasse!' az keyvault secret set \ --vault-name kv-mon-app \ --name api-key \ --value 'sk-abc123...'</pre>	RBAC Authorization = permissions fines via IAM Azure Soft-delete : protection contre suppression accidentelle Ne jamais mettre de secrets dans Git ou ConfigMap Auditer les accès : <code>az keyvault show --show-deleted</code>
Activer Workload Identity sur AKS	<pre># Activer sur le cluster az aks update \ --resource-group rg-prod \ --name aks-prod \ --enable-workload-identity \ --enable-oidc-issuer # Récupérer l'OIDC issuer URL \$oidcUrl = az aks show -g rg-prod -n aks-prod \ --query oidcIssuerProfile.issuerUrl -o tsv # Créer une Managed Identity az identity create \ --resource-group rg-prod \ --name mi-mon-app # Récupérer les infos \$clientId = az identity show -g rg-prod -n mi-mon-app \ --query clientId -o tsv \$principalId = az identity show -g rg-prod -n mi-mon-app \ --query principalId -o tsv</pre>	Workload Identity = pods s'authentifient via identité managée Pas de secret statique stocké dans le cluster OIDC Issuer = fédération d'identité entre AAD et K8s Remplace l'ancien AAD Pod Identity (déprécié)
Lier l'identité au ServiceAccount	<pre># Créer le ServiceAccount K8s kubectl create serviceaccount mon-app-sa -n mon-app kubectl annotate serviceaccount mon-app-sa \ azure.workload.identity/client-id=\$clientId -n mon-app # Créer une Federated Identity Credential (lien AAD <-> K8s SA) az identity federated-credential create \ --name fc-mon-app \ --identity-name mi-mon-app \ --resource-group rg-prod \ --issuer \$oidcUrl \ --subject system:serviceaccount:mon-app:mon-app-sa # Donner accès au Key Vault az role assignment create \ --role 'Key Vault Secrets User' \ --assignee \$principalId \ --scope \$(az keyvault show -g rg-prod -n kv-mon-app --query id -o tsv)</pre>	Federated Credential = le pod prouve son identité via OIDC Key Vault Secrets User = accès lecture seule aux secrets Pas de mot de passe à gérer, rotation automatique Audit trail complet dans Azure Monitor

Étape / Composant	Commande / YAML	Description
Méthode A : CSI Secret Store Driver (montage en volume)	<pre># Installer le driver helm repo add csi-secrets-store-provider-azure \ https://azure.github.io/secret s-store-csi-driver-provider-azure/charts helm install csi csi-secrets-store-provider-azure/csi-secrets-store-provider-azure \ --namespace kube-system # SecretProviderClass apiVersion: secrets-store.csi.x-k8s.io/v1 kind: SecretProviderClass metadata: name: azure-kv-provider namespace: mon-app spec: provider: azure parameters: usePodIdentity: 'false' clientID: '<workload-identity-client-id>' keyvaultName: kv-mon-app tenantId: '<tenant-id>' objects: array: - objectName: db-password objectType: secret objectVersion: '' - objectName: api-key objectType: secret secretObjects: - secretName: app-secrets type: Opaque data: - objectName: db-password key: DB_PASSWORD - objectName: api-key key: API_KEY # Utilisation dans le Deployment volumes: - name: secrets-vol csi: driver: secrets-store.csi.k8s.io readOnly: true volumeAttributes: secretProviderClass: azure-kv-provider</pre>	CSI Driver = montage des secrets comme fichiers ET/OU comme variables d'environnement (secretObjects) Avantage : rotation automatique visible dans le pod Montage en /mnt/secrets-store/ Nécessite le pod de redémarrer pour voir la rotation
Méthode B : External Secrets Operator (synchronisation K8s Secrets)	<pre># Installer ESO helm repo add external-secrets https://charts.external-secrets.io helm install external-secrets external-secrets/external-secrets \ --namespace external-secrets --create-namespace # ClusterSecretStore (une seule fois pour tout le cluster) apiVersion: external-secrets.io/v1beta1 kind: ClusterSecretStore metadata: name: azure-keyvault-store spec: provider: azurekv: authType: WorkloadIdentity vaultUrl: https://kv-mon-app.vault.azure.net serviceAccountRef: name: external-secrets-sa namespace: external-secrets # ExternalSecret par namespace apiVersion: external-secrets.io/v1beta1 kind: ExternalSecret metadata: name: app-secrets namespace: mon-app spec: refreshInterval: 1h secretStoreRef: name: azure-keyvault-store kind: ClusterSecretStore target: name: app-secrets creationPolicy: Owner data: - secretKey: DB_PASSWORD remoteRef: key: db-password - secretKey: API_KEY remoteRef: key: api-key</pre>	ESO crée et synchronise des Secrets K8s natifs Les pods utilisent des env vars classiques (pas de volume) Refresh : toutes les heures par défaut Pas de redémarrage nécessaire pour la rotation Plus flexible que CSI driver pour l'injection en env vars
Rotation & Audit des Secrets	<pre># Voir les secrets dans Key Vault az keyvault secret list --vault-name kv-mon-app -o table # Historique des versions d'un secret az keyvault secret list-versions \ --vault-name kv-mon-app \ --name db-password -o table # Activer la rotation automatique (Function Azure) # Event Grid → Key Vault → Azure Function → Rotation # Audit des accès az monitor log-analytics query \ --workspace \$wsId \ --analytics-query 'AzureDiagnostics where ResourceType == "VAULTS" where OperationName == "SecretGet"' # Forcer le refresh ESO kubectl annotate externalsecret app-secrets \ force-sync=\$(date +%s) -n mon-app</pre>	Rotation : Event Grid déclenche une Azure Function Audit : activer Diagnostic Settings sur Key Vault Log Analytics : tracer chaque accès aux secrets Alerte si un secret expire dans < 30 jours Politique d'expiration recommandée : 90 jours

Étape / Composant	Commande / YAML	Description
Sealed Secrets (secrets chiffrés dans Git)	<pre># Installer Sealed Secrets controller helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets helm install sealed-secrets sealed-secrets/sealed-secrets \ --namespace kube-system # Installer kubeseal CLI choco install kubeseal # Windows # Créer un SealedSecret (chiffrement côté client) kubectl create secret generic mon-secret \ --from-literal=password='S3cr3t!' \ --dry-run=client -o yaml \ kubeseal --controller-name=sealed-secrets \ --controller-namespace=kube-system \ --format yaml > sealed-secret.yaml # Ce fichier peut être commité dans Git en sécurité ! git add sealed-secret.yaml git commit -m 'add sealed secret for mon-app'</pre>	Sealed Secrets = chiffrement asymétrique des secrets Seul le controller dans le cluster peut déchiffrer Fichier sealed peut être commité dans Git (GitOps) Roter la clé de chiffrement régulièrement Compatible avec ArgoCD/Flux

■ *Recommandation : utiliser Azure Key Vault + External Secrets Operator (ESO) avec Workload Identity comme solution principale. Sealed Secrets pour les configurations GitOps. Ne jamais stocker de secrets en clair dans les YAML ou les variables de pipeline.*